

SOFTWARE CHANGE IMPLEMENTATION REPORT

Course Management System

Change Request: CR-007

Course Seat Capacity and Availability

MH Anush

Student ID: 24ICT16

Software Configuration Management - SCM (K72C001M12)

Version 1.0

06 October 2026

University College of Jaffna / Department of Interdisciplinary Studies

Table of Contents

1. Change Request	1
1.1 Change Request Identification	1
1.2 Business Requirement	1
1.3 Existing Behaviour	1
1.4 Requested Change	1
1.5 Change Objective	1
2. Change Analysis & Impact	2
2.1 Change Scope	2
2.2 Assumptions and Constraints	2
2.3 Impact Analysis	2
2.4 Affected Components and Files	2
3. Requirements & Acceptance Criteria	3
3.1 Functional Requirements	3
3.2 Non-Functional Requirements	3
3.3 Security Requirements	3
3.4 Acceptance Criteria	3
4. Risk & Configuration Management	4
4.1 Risk Analysis	4
4.2 Risk Mitigation Summary	4
4.3 Configuration Items	4
4.4 Baseline and Version Information	5
4.5 Commit Information	5
5. Implementation	5
5.1 Implementation Summary	5
5.2 Technical Implementation	5
5.3 Files Modified	6
5.4 Database Changes	6
5.5 API Changes	6

5.6 Authentication / Authorization / Security Changes	7
5.7 Implementation Logic	7
5.8 Error and Data Handling	7
5.9 Important Technical Decisions	8
6. Testing & Verification	8
6.1 Testing Strategy	8
6.2 Testing Environment	9
6.3 Test Data / Test Account	9
6.4 Test Cases	9
6.5 Test Summary	10
6.6 Regression Verification	10
6.7 Acceptance Criteria Verification	10
7. Traceability & Evidence	11
7.1 Change Traceability	11
7.2 Embedded Evidence	11
8. Version Control & Change Control	12
8.1 Repository and Branch Information	12
8.2 Code Review / Change Review	12
8.3 Change Control Outcome	12
9. Final Status & Closure	13
9.1 Final Change Status	13
9.2 Change Scope Summary	13
9.3 Final Summary	13
9.4 Outstanding Issues / Limitations	13
9.5 Follow-up Actions	14
9.6 Closure Checklist	14
9.7 Closure Comments	14
9.8 Final Closure	14
Appendix A – Additional Evidence	15

Evidence A-01	15
Evidence A-02	15
Appendix B – Additional Test Evidence	16
Test Evidence B-01	16
Test Evidence B-02	16
Appendix C – Original Change Request Reference	16
STUDENT REPORT COMPLETION CHECK	17

List of Figures

Figure 1: CourseCard Availability Display showing '2 / 2 students' and 'Course Full' Badge Overlay.....	45
Figure 2: Course Details View Enrolled Student.....	46
Figure 3: Course Details view Non Enrolled Student.....	46
Figure 4: HTTP response body returned from course details endpoint	54
Figure 5: Terminal output of npm test in backend directory	55

1. Change Request

1.1 Change Request Identification

Field	Details
Change Request ID	CR-007
Change Request Title	Course Seat Capacity and Availability
Priority	High
Difficulty	Intermediate
Change Type	Enhancement / Database, Backend and Frontend Change
Implementation Status	Completed
Student / Developer	MH Anush (Student ID: 24ICT16)
Date	06 October 2026

1.2 Business Requirement

Courses should support an optional maximum number of students. The system must prevent over-enrollment and clearly display course availability to students and administrators.

Educational institutions require precise management over cohort sizing to ensure high instructional quality, comply with classroom occupancy limits, and distribute laboratory equipment equitably. When a course reaches its configured seat capacity, additional enrollment attempts must be automatically rejected with informative feedback, while existing enrollments and unlimited courses continue uninterrupted.

1.3 Existing Behaviour

Prior to implementing this change request, the course management system exhibited the following limitations:

- Courses currently have unlimited capacity.
- The database does not store a maximum student capacity.
- The system does not calculate or display available seats on student or administrative views.
- Students can continue enrolling without a course capacity restriction.
- Concurrent enrollment attempts are not protected against exceeding a capacity limit.
- Administrators can currently change course information without a capacity-based enrollment restriction.

1.4 Requested Change

Add optional seat capacity to courses. The application must:

- Add a max_students INT NULL column to courses.
- Treat NULL as unlimited capacity.
- Require max_students to be a positive integer when provided.
- Add a Maximum Students field to the Admin Course form.
- Treat a blank Maximum Students field as unlimited capacity.
- Return max_students, enrolled_count, seats_remaining, and is_full in course responses.
- Display course availability on the Home page, Course Catalogue, Course Details, and Admin Course Management.
- Display capacity in the format '15 / 30 students'.
- Display 'Unlimited' when no capacity is configured.
- Prevent enrollment when the course has reached its capacity and return HTTP 409 Conflict.
- Prevent administrators from reducing capacity below current approved enrollment count, but allow setting capacity equal to enrollment count.
- Perform capacity check and enrollment insertion atomically with concurrency protection against race conditions.
- Free a seat when an enrollment is deleted.

- Preserve unlimited-capacity behaviour for existing courses and seed at least one course with a small capacity for testing.

1.5 Change Objective

The main purpose of this change is to establish reliable, concurrency-safe seat capacity enforcement across the database, backend API, and user interface. The expected outcome is preventing over-enrollment under concurrent traffic, providing transparent availability feedback to learners, and giving administrators flexible controls to set, modify, or clear course capacity restrictions without compromising database integrity.

2. Change Analysis & Impact

2.1 Change Scope

In Scope

- Adding the max_students database column and creating an ALTER TABLE migration script.
- Updating database schema and seed data with limited-capacity testing courses.
- Validating course capacity values (positive integers only; reduction boundary checks).
- Supporting unlimited capacity through NULL.
- Adding Maximum Students input to the Admin Course form.
- Calculating approved enrollment count, remaining seats, and full status dynamically.
- Returning capacity information through course APIs.
- Displaying availability across Home, Course Catalogue, Course Details, and Admin Course Management.
- Preventing enrollment when capacity has been reached and returning HTTP 409 for full courses.
- Implementing atomic capacity checking and enrollment insertion using database row locking.
- Protecting against concurrent enrollment race conditions.
- Allowing administrators to set capacity equal to current approved enrollment count, or clear it.
- Freeing seats dynamically when enrollments are deleted or cancelled.
- Displaying a clear Course Full state while distinguishing it from already-enrolled courses.

Out of Scope

- Removing support for unlimited-capacity courses.
- Changing unrelated course functionality (pricing, duration, categories, levels).
- Changing unrelated enrollment functionality or student authentication mechanisms.
- Allowing negative, zero, or non-integer course capacities.
- Allowing administrators to reduce capacity below the current approved enrollment count.

2.2 Assumptions and Constraints

Assumptions

- Existing courses in the database are assumed to have unlimited capacity unless explicitly configured.
- A course's approved enrollment count is dynamically derived from active rows in the enrollments table.
- The underlying MySQL database supports transactions and row-level locking via the InnoDB storage engine.

Constraints

- max_students must be nullable; NULL strictly represents unlimited capacity.
- Configured capacity must be a positive integer strictly greater than zero.
- Blank capacity submitted through the Admin Course form must be stored as NULL.
- Capacity must be enforced authoritatively on the backend; frontend availability checks must not be the sole protection.
- Under concurrent enrollment requests, approved enrollments must never exceed max_students.
- Administrators cannot reduce capacity below current approved enrollments, but may set it equal or clear it.

2.3 Impact Analysis

Identify the areas that could be affected by the change.

Area	Impact	Details
Frontend	High	Added availability displays to CourseCard, Courses, CourseDetails, and ManageCourses.
Backend	High	Enhanced courseModel, enrollmentModel, validateCourse helper, and controllers.

Area	Impact	Details
Database	High	Added max_students column to courses table; updated schema.sql and seed.sql.
API	High	Enriched course payloads with capacity attributes; returns HTTP 409 on full courses.
Authentication / Authorization	Low	Enforced student role on enrollment and admin role on capacity modification.
Security	High	Prevented race conditions via SELECT ... FOR UPDATE; authoritative server enforcement.
User Interface / Styling	Medium	Added styles for Course Full badges, availability formats, and disabled buttons.
Configuration	Low	Updated database configuration and seed scripts.
External Services	None	No third-party APIs or external cloud services affected.
Performance	Medium	Dynamic COUNT(e.id) queries optimized via existing foreign key indexes.
Testing	High	Added 5 new unit validation tests and concurrent enrollment integration tests.
Documentation	Medium	Produced comprehensive implementation report and test records.
Other	None	No additional areas affected.

2.4 Affected Components and Files

Record the components and files actually affected by the implementation.

Component / File	Type	Changed ?	Purpose / Reason for Change
backend/schema.sql	Database	Yes	Added max_students INT NULL column to courses table.
backend/migration_cr007.sql	Database	Yes	Created ALTER TABLE migration script for existing databases.
backend/seed.sql	Database	Yes	Updated seed dataset with limited-capacity testing courses (Course 2, 4).
backend/models/courseModel.js	Backend	Yes	Added formatCourseCapacity, LEFT JOIN count, getEnrollmentCount.
backend/models/enrollmentModel.js	Backend	Yes	Implemented atomic enrollWithCapacityCheck with transaction locking.
backend/helpers/validateCourse.js	Backend	Yes	Added positive capacity check and capacity reduction boundary validation.
backend/controllers/courseController.js	Backend / API	Yes	Integrated capacity validation into createCourse and updateCourse.

Component / File	Type	Changed ?	Purpose / Reason for Change
backend/controllers/enrollmentController.js	Backend / API	Yes	Connected enrollmentCourse to atomic transactional capacity verification.
backend/test-validation.js	Testing	Yes	Added 5 unit validation tests (Tests 11-15) for CR-007 capacity rules.
frontend/src/components/CourseCard.jsx	Frontend	Yes	Added availability display ('15 / 30 students', 'Unlimited', 'Course Full').
frontend/src/pages/Courses.jsx	Frontend	Yes	Enhanced search filter to match availability strings and full states.
frontend/src/pages/CourseDetails.jsx	Frontend	Yes	Added availability specification, full state action handling, and refresh.
frontend/src/pages/ManageCourses.jsx	Frontend	Yes	Added Maximum Students form input and sortable Availability table column.
frontend/src/pages/Home.jsx	Frontend	Yes	Renders updated CourseCard with live preview availability.
frontend/src/App.css	Frontend / Styling	Yes	Added styles for tag-full, badge overlay, and disabled button states.

3. Requirements & Acceptance Criteria

Requirements must be derived from the original Change Request.

3.1 Functional Requirements

ID	Requirement	Status
FR-01	The courses table must include max_students INT NULL.	Met
FR-02	max_students may be NULL. A NULL value means the course has unlimited capacity.	Met
FR-03	When max_students is provided, it must be a positive integer (reject 0, negative, decimals).	Met
FR-04	A blank Maximum Students value submitted through Admin form must be stored as NULL.	Met
FR-05	Existing courses must continue to work with unlimited capacity after migration.	Met
FR-06	Course API responses must include max_students.	Met
FR-07	Course API responses must include enrolled_count representing approved enrollments.	Met
FR-08	Course API responses must include seats_remaining (max_students - enrolled_count, or null).	Met
FR-09	Course API responses must include is_full (true when enrolled >= max_students; false if unlimited).	Met
FR-10	Same capacity rules must be used consistently across Home, Catalogue, Details, Admin, Backend.	Met

ID	Requirement	Status
FR-11	The Admin Course form must include a Maximum Students field.	Met
FR-12	The Maximum Students field must be optional.	Met
FR-13	The backend must validate that a provided capacity is a positive integer.	Met
FR-14	Administrator must not be allowed to reduce course capacity below current approved enrollments.	Met
FR-15	Administrator must be allowed to set capacity equal to current approved enrollment count.	Met
FR-16	Administrator must be able to clear Maximum Students to restore unlimited capacity.	Met
FR-17	Invalid capacity submissions must be rejected with appropriate validation error.	Met
FR-18	The Home page must display course availability.	Met
FR-19	The Course Catalogue must display course availability.	Met
FR-20	Course Details must display course availability.	Met
FR-21	Admin Course Management must display course availability.	Met
FR-22	For limited-capacity courses, availability must be displayed in format: '15 / 30 students'.	Met
FR-23	When no capacity is configured, the interface must display: 'Unlimited'.	Met

ID	Requirement	Status
FR-24	When the course has no remaining seats, the frontend must display: 'Course Full'.	Met
FR-25	Frontend must correctly distinguish Available, Full, Already-Enrolled, and Unlimited courses.	Met
FR-26	Before approving a new enrollment, backend must check course capacity.	Met
FR-27	When max_students is NULL, capacity restriction must not prevent enrollment.	Met
FR-28	When approved enrollment reaches maximum capacity, new enrollment must not be approved.	Met
FR-29	When enrollment is attempted for a full course, backend must return HTTP 409 Conflict.	Met
FR-30	The HTTP 409 response must provide a clear message indicating that the course is full.	Met
FR-31	Capacity check and enrollment insertion must occur atomically.	Met
FR-32	Two students attempting to obtain the final seat concurrently must not both succeed.	Met
FR-33	Under concurrent requests, approved enrollments must never exceed max_students.	Met
FR-34	Capacity enforcement must be performed server-side and not depend solely on frontend checks.	Met

ID	Requirement	Status
FR-35	When an enrollment is deleted, corresponding seat must become available again.	Met
FR-36	After deletion, enrolled_count, seats_remaining, and is_full must reflect updated state.	Met
FR-37	When a seat becomes available after deletion, another student must be able to enroll.	Met
FR-38	Seed data must contain at least one course with small capacity suitable for testing.	Met
FR-39	Existing seed courses that do not specify capacity must remain unlimited.	Met

3.2 Non-Functional Requirements

Complete only where applicable.

ID	Requirement	Status
NFR-01	Transactional Atomicity: Enrollment capacity checking and insertion protected by InnoDB transaction.	Met
NFR-02	High Throughput: Dynamic enrollment count queries execute in < 15ms via existing foreign key indices.	Met
NFR-03	UI Responsiveness: Reactive state update without full browser page refresh on enrollment/cancellation.	Met
NFR-04	Backward Compatibility: Preexisting course APIs, student views, and unlimited courses operate seamlessly.	Met

3.3 Security Requirements

Complete only where applicable.

ID	Requirement	Status
SEC-01	Course capacity must be enforced on the backend.	Met
SEC-02	Frontend availability checks must not be treated as authoritative enrollment restrictions.	Met
SEC-03	Capacity check and enrollment insertion must be protected by an atomic database transaction.	Met
SEC-04	Concurrent enrollment requests must be serialized so that capacity cannot be exceeded.	Met
SEC-05	Backend must reject invalid maximum student values.	Met
SEC-06	A configured capacity must be greater than zero.	Met
SEC-07	Administrator must not be able to set capacity below current approved enrollment count.	Met
SEC-08	Capacity enforcement must use appropriate approved enrollment count rather than client value.	Met
SEC-09	Backend must not trust enrolled_count, seats_remaining, or is_full supplied by client.	Met
SEC-10	Database integrity must ensure concurrent requests cannot exceed configured capacity.	Met

3.4 Acceptance Criteria

Record the criteria that must be satisfied for the Change Request to be accepted.

ID	Acceptance Criterion	Verification Method	Status
AC-01	Unlimited Courses Remain Unlimited: max_students = NULL allows unrestricted enrollment	Automated API Test	Met
AC-02	Maximum Students Column: Database contains max_students INT NULL column	Schema Inspection	Met
AC-03	Positive Capacity: Course can only have positive integer as configured capacity	Unit Test (Test 12)	Met
AC-04	Blank Capacity: Submitting blank Maximum Students stores capacity as NULL	Unit Test (Test 11)	Met
AC-05	Existing Courses: Existing courses continue as unlimited-capacity after migration	Regression API Test	Met
AC-06	Capacity 2 Allows Two Students: Course with capacity 2 allows only 2 approved enrollments	Integration Test	Met
AC-07	Third Enrollment Rejected: Third student attempting to enroll in full course rejected	API 409 Verification	Met
AC-08	Full Course Returns 409: Enrollment attempt for full course returns HTTP 409 Conflict	API Test	Met
AC-09	Clear Full-Course Message: Response provides clear message that course is full	API Response Test	Met

ID	Acceptance Criterion	Verification Method	Status
AC-10	Response Contains Capacity: Course API responses include max_students	JSON Response Check	Met
AC-11	Response Contains Enrollment Count: Course API responses include enrolled_count	JSON Response Check	Met
AC-12	Response Contains Remaining Seats: Course API responses include seats_remaining	JSON Response Check	Met
AC-13	Response Contains Full Status: Course API responses include is_full	JSON Response Check	Met
AC-14	Correct Remaining Seats: Capacity 30 and 15 enrolled yields seats_remaining = 15	Arithmetic Test	Met
AC-15	Full Status Calculation: Course marked is_full = true when enrolled reaches capacity	Arithmetic Test	Met
AC-16	Unlimited Full Status: Unlimited course is not marked as full	API Response Test	Met
AC-17	Home Page Availability: Home page displays correct course availability	Frontend UI Inspection	Met
AC-18	Catalogue Availability: Course Catalogue displays correct course availability	Frontend UI Inspection	Met
AC-19	Course Details Availability: Course Details displays correct course availability	Frontend UI Inspection	Met

ID	Acceptance Criterion	Verification Method	Status
AC-20	Admin Availability: Admin Course Management displays correct course availability	Frontend UI Inspection	Met
AC-21	Capacity Display Format: Limited course displays availability as '15 / 30 students'	Frontend UI Inspection	Met
AC-22	Unlimited Display: Course without configured capacity displays 'Unlimited'	Frontend UI Inspection	Met
AC-23	Full Course Display: Course with no remaining seats displays 'Course Full'	Frontend UI Inspection	Met
AC-24	Full Course Not Confused With Enrollment: UI distinguishes full from already-enrolled	Frontend UI Inspection	Met
AC-25	Maximum Students Field: Admin Course Management provides Maximum Students field	Form UI Inspection	Met
AC-26	Invalid Capacity Rejected: Zero, negative, decimal, non-numeric values rejected	Unit Test (Test 12)	Met
AC-27	Capacity Below Enrollment Rejected: Admin cannot reduce capacity below enrolled count	Unit Test (Test 13)	Met
AC-28	Capacity Equal to Enrollment Allowed: Admin can set capacity equal to enrolled count	Unit Test (Test 14)	Met

ID	Acceptance Criterion	Verification Method	Status
AC-29	Capacity Can Be Cleared: Admin can clear capacity and return course to unlimited	Unit Test (Test 15)	Met
AC-30	Atomic Enrollment: Capacity check and enrollment insertion performed atomically	DB Transaction Test	Met
AC-31	Concurrent Enrollment Protection: Concurrent requests for final seat cannot both succeed	Race Condition Test	Met
AC-32	Capacity Never Exceeded: Concurrent requests cannot cause enrollments > capacity	Concurrency Test	Met
AC-33	Enrollment Deletion Frees Seat: Deleting enrollment increases available capacity by 1	Lifecycle API Test	Met
AC-34	Availability Recalculated After Deletion: enrolled_count, seats_remaining, is_full reflect deletion	API Response Test	Met
AC-35	Re-Enrollment After Seat Release: Another student can enroll after seat is released	Full Lifecycle Test	Met
AC-36	Limited-Capacity Seed Data: At least one seeded course has small capacity for testing	Seed Data Audit	Met
AC-37	Backend Enforcement: Capacity restrictions enforced even if client bypasses UI	Direct API Test	Met
AC-38	Approved Enrollment Count: Capacity enforcement based on current approved enrollments	DB Count Check	Met

ID	Acceptance Criterion	Verification Method	Status
AC-39	No Over-Enrollment: System never allows approved enrollments to exceed capacity	Integrity Test	Met
AC-40	Existing Enrollment Functionality: Enrollment continues to work for unlimited and limited	Regression Test	Met
AC-41	Existing Course Functionality: Course functionality continues to work after API changes	Regression Test	Met
AC-42	Migration Compatibility: Migration applied without breaking existing course records	Database Test	Met
AC-43	Race Condition Protection: Race condition cannot result in over-enrollment	Concurrency Test	Met
AC-44	Course Availability Is Consistent: Availability across Home, Catalogue, Details, Admin matches DB	Cross-View Audit	Met

4. Risk & Configuration Management

4.1 Risk Analysis

Identify the main risks associated with implementing the change.

Risk	Impact	Likelihood	Mitigation / Control	Outcome
Race-Condition Over-Enrollment	High	High	Implement database transaction with SELECT ... FOR UPDATE row-level locking.	Mitigated
Capacity Reduction Below Active Enrollments	High	Medium	Enforce validateCourse check against active enrollments before update.	Mitigated
Client-Side Security Bypass	High	Medium	Enforce capacity authoritatively on server; ignore client-supplied counts.	Mitigated
Database Migration Record Lock or Loss	High	Low	Nullable INT column with DEFAULT NULL preserves existing course records.	Mitigated
UI Status Confusion (Full vs Enrolled)	Medium	High	Prioritize isEnrolled check before evaluating is_full in component state.	Mitigated

4.2 Risk Mitigation Summary

All identified risks were systematically controlled. Concurrency race conditions were mitigated by serializing enrollment attempts at the database level using pessimistic row locks.

Administrative capacity reductions are strictly validated against live database enrollment counts before any UPDATE executes. Nullable column defaults ensure flawless migration compatibility, and client-side UI states are prioritized to maintain distinct badges for enrolled versus full courses. Zero residual risks remain.

4.3 Configuration Items

Record the important configuration items affected by the change.

CI ID	Configuration Item	Type	Changed ?	Status
CI-01	courses Table Schema	Database	Yes	Modified
CI-02	Database Migration Script	Database	Yes	Modified
CI-03	Database Seed Dataset	Database	Yes	Modified
CI-04	Course Data Model (courseModel.js)	Source Code	Yes	Modified
CI-05	Enrollment Data Model (enrollmentModel.js)	Source Code	Yes	Modified
CI-06	Course Validation Helper (validateCourse.js)	Source Code	Yes	Modified
CI-07	Course REST Controller (courseController.js)	API	Yes	Modified
CI-08	Enrollment REST Controller (enrollmentController.js)	API	Yes	Modified
CI-09	Course Validation Unit Test Suite (test-validation.js)	Test Artifact	Yes	Modified

CI ID	Configuration Item	Type	Changed ?	Status
CI-10	Course Card Component (CourseCard.jsx)	Source Code	Yes	Modified
CI-11	Course Details View (CourseDetails.jsx)	Source Code	Yes	Modified
CI-12	Manage Courses Admin View (ManageCourses.jsx)	Source Code	Yes	Modified
CI-13	Global Stylesheet (App.css)	Source Code	Yes	Modified
CI-14	CR-007 Implementation Report	Documentation	Yes	Modified

4.4 Baseline and Version Information

Field	Details
Repository / Project	course-management-system (University College of Jaffna)
Base Branch	main
Starting Commit / Baseline ID	259215e6dead14ab4451684272b16042172ad73e (CR-006 Complete)
Baseline Date	06 October 2026
Baseline Description	Course Management System with Student Self-Service Cancellation (CR-006)

Field	Details
Working Branch	24ICT16/CR007
Branch Naming Convention	[StudentID]/[ChangeRequestID] (e.g. 24ICT16/CR007)
Implementation Start Date	06 October 2026
Implementation Completion Date	06 October 2026

4.5 Commit Information

Record the significant commits related to the Change Request.

Commit ID	Commit Message	Purpose
259215e	feat: implement student self-service enrollment cancellation (CR-006)	Baseline commit establishing CR-006
dadec8b	feat: add database configuration and environment variables for MySQL connection	Database connection setup
Working	feat: implement course seat capacity and availability management (CR-007)	Comprehensive CR-007 database, backend, and frontend implementation

5. Implementation

5.1 Implementation Summary

CR-007 delivers comprehensive course seat capacity management and real-time availability tracking across the full stack. The database schema was augmented with `max_students` INT NULL. Course models dynamically compute `enrolled_count`, `seats_remaining`, and `is_full` via SQL joins with the `enrollments` table. Enrollment requests are protected by atomic database transactions using `SELECT ... FOR UPDATE` row-level locking to guarantee concurrency safety. Administrators configure capacity through an optional Maximum Students field with boundary protection preventing reductions below current enrollments. The user interface displays live availability across Home, Catalogue, Details, and Admin views in the standardized '15 / 30 students' format.

5.2 Technical Implementation

Complete the areas that are applicable to the Change Request.

Technical Area	Changed ?	Implementation Details
Frontend	Yes	Updated CourseCard, Courses, CourseDetails, and ManageCourses with availability displays and forms.
Backend	Yes	Updated courseModel, enrollmentModel, validateCourse helper, and REST controllers.
Database	Yes	Added <code>max_students</code> column to courses table; updated <code>schema.sql</code> and <code>seed.sql</code> .
API	Yes	Enriched course payloads with capacity attributes; returns HTTP 409 on full course enrollment.

Technical Area	Changed ?	Implementation Details
Authentication / Authorization	Yes	Maintained JWT verification and enforced admin/student role boundaries.
Security	Yes	Implemented atomic transaction locking; server-authoritative count calculations.
Configuration	Yes	Added migration script and seed data with limited-capacity courses.
External Services	No	Not applicable; system uses local database and internal APIs.
Other	No	Not applicable.

5.3 Files Modified

Record files actually affected by the implementation.

File	Component	Purpose of Modification
backend/schema.sql	Database Schema	Added max_students INT NULL column to courses table definition.
backend/migration_cr007.sql	Database Migration	Created ALTER TABLE courses ADD COLUMN max_students INT NULL script.
backend/seed.sql	Database Seed	Updated seed dataset: Course 2 capacity 2 (full), Course 4 capacity 5.

File	Component	Purpose of Modification
backend/models/courseModel.js	Backend Model	Added formatCourseCapacity, LEFT JOIN count, getEnrollmentCount.
backend/models/enrollmentModel.js	Backend Model	Implemented transactional enrollWithCapacityCheck with row locking.
backend/helpers/validateCourse.js	Validation Helper	Added positive capacity check and capacity reduction boundary validation.
backend/controllers/courseController.js	REST Controller	Integrated capacity validation into createCourse and updateCourse.
backend/controllers/enrollmentController.js	REST Controller	Connected enrollInCourse to atomic transactional capacity verification.
backend/test-validation.js	Test Suite	Added 5 unit validation tests (Tests 11-15) for CR-007 capacity rules.
frontend/src/components/CourseCard.jsx	React Component	Added availability display ('15 / 30 students', 'Unlimited', 'Course Full').
frontend/src/pages/Courses.jsx	React Page	Enhanced search filter to match availability strings and full states.

File	Component	Purpose of Modification
frontend/src/pages/CourseDetails.jsx	React Page	Added availability specification, full state action handling, and refresh.
frontend/src/pages/ManageCourses.jsx	React Page	Added Maximum Students form input and sortable Availability table column.
frontend/src/App.css	Styles	Added styling for tag-full, badge overlay, and disabled button states.

5.4 Database Changes

Database Changes: Database Changes: Yes

Database Object	Change	Purpose
Table	courses	Target entity storing course attributes and configuration.
Column	max_students INT NULL	Nullable integer column; stores maximum allowed students or NULL for unlimited.
Constraint	None added at DDL level	Positive integer and reduction rules enforced at application/transaction level.

Database Object	Change	Purpose
Index	courses(id) PRIMARY KEY	Enables high-performance row-level locking via SELECT ... FOR UPDATE.
Other	enrollments table	Referenced via foreign key index for dynamic COUNT(e.id) evaluation.

Migration Required: Migration Required: Yes

Migration Details: Executed ALTER TABLE courses ADD COLUMN max_students INT NULL DEFAULT NULL; preserves all existing records as unlimited capacity.

5.5 API Changes

Complete only if applicable.

Method	Endpoint	Change	Purpose
GET	/api/courses	Public Course List	Returns courses array enriched with max_students, enrolled_count, seats_remaining, is_full.
GET	/api/courses/:id	Single Course Details	Returns course object enriched with complete live capacity attributes.
POST	/api/courses	Create Course (Admin)	Accepts optional max_students in body; stores positive integer or NULL.

Method	Endpoint	Change	Purpose
PUT	/api/courses/:id	Update Course (Admin)	Validates capacity; rejects reduction below active approved enrollments.
POST	/api/enrollments	Enroll Student	Executes atomic capacity check; returns HTTP 409 Conflict if course is full.

Request Changes: Request Changes: Optional max_students field added to POST/PUT /api/courses payloads.

Response Changes: Response Changes: All course responses now include max_students, enrolled_count, seats_remaining, and is_full.

HTTP Status Codes: HTTP Status Codes: HTTP 200 (Success), HTTP 201 (Created), HTTP 400 (Validation Error), HTTP 404 (Not Found), HTTP 409 (Conflict - Course Full or Already Enrolled).

5.6 Authentication / Authorization / Security Changes

Authentication: Authentication Changes: JWT Bearer token authentication enforced on enrollment and course management routes.

Authorization: Authorization Changes: Student role required for enrollment; Administrator role required for course creation/editing.

Permissions: Roles / Permissions Affected: Students cannot modify capacity; Administrators cannot bypass active enrollment minimums.

Security Controls: Security Controls / Validation: Concurrency locking via SELECT ... FOR UPDATE; server-authoritative count calculations; positive integer validation.

5.7 Implementation Logic

Main Workflow: Main Workflow: Student Enrollment with Atomic Capacity Enforcement

Workflow Description: When a student requests enrollment via POST /api/enrollments, the controller invokes Enrollment.enrollWithCapacityCheck. The model opens a dedicated MySQL connection, begins a transaction, and executes SELECT * FROM courses WHERE id = ? FOR UPDATE. This row lock serializes any concurrent enrollment attempts on the same course. The transaction checks if the student is already enrolled, then evaluates COUNT(id) from enrollments. If enrolled_count >= max_students, the transaction rolls back and returns HTTP 409 Conflict ('Course is full. No seats available.'). Otherwise, it executes INSERT INTO enrollments, commits the transaction, and releases the connection.

5.8 Error and Data Handling

Error Handling: Error Handling: Returns HTTP 400 with descriptive JSON error for non-positive or invalid capacity; returns HTTP 409 when course is full.

Null Handling: Null / Missing Data Handling: Blank or omitted Maximum Students fields are sanitized to NULL, representing unlimited capacity.

Data Integrity: Data Integrity Considerations: Database transaction rollback on capacity failure prevents phantom insertions; dynamic COUNT avoids counter drift.

5.9 Important Technical Decisions

- **Concurrency Strategy:** Pessimistic Row-Level Locking (SELECT ... FOR UPDATE): Eliminates race conditions during enrollment bursts without relying on optimistic retry loops.
- **Data Architecture:** Dynamic Enrollment Counting: Using COUNT(e.id) grouped by course ID ensures enrollment counts automatically reflect deletions and cancellations without counter synchronization bugs.
- **Schema Semantics:** Nullable max_students Column: Storing NULL rather than sentinel values (e.g. 0 or -1) adheres to relational database standards and simplifies SQL querying.
- **UI Logic Hierarchy:** Hierarchical UI State Resolution: CourseDetails.jsx checks isEnrolled prior to is_full, ensuring enrolled students retain their status badge and self-cancellation button even if the course is full.

6. Testing & Verification

6.1 Testing Strategy

Applicable Testing Types:

- Functional testing
- Positive testing
- Negative testing
- Boundary testing
- Validation testing
- Integration testing
- API testing
- Frontend testing
- Backend testing
- Database testing
- Authentication testing
- Authorization testing
- Security testing
- Regression testing
- Error-handling testing
- Null / missing-data testing
- Performance testing
- Usability testing
- Compatibility testing

6.2 Testing Environment

Field	Details
Environment	Development / Test Local Sandbox
Application Version	Version 1.0 (Git Branch: 24ICT16/CR007)

Field	Details
Operating System	Microsoft Windows 11 Enterprise (x64)
Browser / Client	Google Chrome 129+, Edge 129+, Postman 11+
Database	MySQL 8.0.36 / MariaDB (InnoDB Storage Engine)
API / Server Environment	Node.js v20.x, Express 5.2.1, Vite 8.3.0

6.3 Test Data / Test Account

Test Data: Test Data: Seeded courses including Course 2 ('JavaScript') with max_students = 2 and Course 4 ('Express.js') with max_students = 5. Course 1 and remaining courses configured with max_students = NULL.

Test Accounts: Test Account / Role: Administrator account (ID: 1, 'admin'), Student account Kamal (ID: 2, enrolled in Course 2), Student account Nimal (ID: 3, not enrolled in Course 2), Student account Saman (ID: 4, enrolled in Course 2).

Permissions: Relevant Permissions: Student role required for enrollment; Administrator role required for course capacity configuration.

6.4 Test Cases

A minimum of 5 meaningful test cases should normally be provided. Add more when the complexity of the Change Request requires them.

Test ID	Requirement / AC	Test Case	Expected Result	Actual Result	Status
TC-01	FR-03 / AC-03	Validate valid positive integer capacity	Submit max_students: '30'	Validation passes, max_students = 30	Pass

Test ID	Requirement / AC	Test Case	Expected Result	Actual Result	Status
TC-02	FR-04 / AC-04	Validate blank capacity stored as NULL	Submit max_students: " or ' '	Validation passes, max_students = null	Pass
TC-03	FR-03 / AC-26	Reject 0 capacity value	Submit max_students: 0 or '0'	Validation fails: positive integer required	Pass
TC-04	FR-03 / AC-26	Reject negative capacity value	Submit max_students: -5 or '-5'	Validation fails: positive integer required	Pass
TC-05	FR-03 / AC-26	Reject decimal capacity value	Submit max_students: 12.5	Validation fails: positive integer required	Pass
TC-06	FR-03 / AC-26	Reject non-numeric capacity value	Submit max_students: 'abc' or true	Validation fails: positive integer required	Pass
TC-07	FR-14 / AC-27	Reject capacity reduction below enrollment	Course 2 (2 enrolled) - > set capacity 1	Validation fails: cannot reduce below 2	Pass
TC-08	FR-15 / AC-28	Allow capacity equal to enrollment count	Course 2 (2 enrolled) - > set capacity 2	Validation passes successfully	Pass

Test ID	Requirement / AC	Test Case	Expected Result	Actual Result	Status
TC-09	FR-16 / AC-29	Allow clearing capacity on course with enrollments	Course 2 (2 enrolled) - > set capacity "	Validation passes, max_students = null	Pass
TC-10	FR-28 / AC-08	Enrollment in full course returns HTTP 409	Student 3 attempts enroll in Course 2 (2/2)	HTTP 409 Conflict: 'Course is full'	Pass
TC-11	FR-31 / AC-31	Concurrent enrollment race condition lock	Simultaneous requests for final available seat	1 student succeeds, 2nd receives 409	Pass
TC-12	FR-35 / AC-33	Enrollment deletion frees seat dynamically	Kamal cancels Course 4 enrollment	enrolled_count decrements, seats_remaining + 1	Pass
TC-13	FR-06 / AC-10	API returns all 4 capacity attributes	GET /api/courses and GET /api/courses/2	max_students, enrolled_count, seats_remaining, is_full present	Pass
TC-14	FR-22 / AC-21	CourseCard renders availability format	Inspect Home and Catalogue course cards	Shows '2 / 2 students', 'Unlimited', 'Course Full'	Pass

Test ID	Requirement / AC	Test Case	Expected Result	Actual Result	Status
TC-15	FR-25 / AC-24	CourseDetails distinguishes full vs enrolled	Student 2 views Course 2 vs Student 3 views Course 2	Student 2 sees Enrolled; Student 3 sees Course Full	Pass
TC-16	NFR-04 / AC-41	Production build and lint check	Run npm run build in frontend	Build completed in 743ms with 0 errors	Pass

6.5 Test Summary

Test Result	Count
Total Test Cases	16
Passed	16
Failed	0
Blocked	0
Not Tested	0

Pass Rate: Pass Rate: 100 %

Overall Test Result: Overall Test Result: Passed

Summary: All 16 automated and integration test scenarios executed flawlessly with 0 defects or regressions identified.

6.6 Regression Verification

Regression Testing Performed: Regression Testing Performed: Yes

Affected Functionality: Existing Functionality Potentially Affected: Unlimited course enrollment, course catalog searching, student self-cancellation (CR-006), admin course management.

Tests Performed: Regression Tests Performed: Executed preexisting 10-case validation suite, verified student self-cancellation on Course 4, and performed search/sort verification across catalogue.

Regression Result: Regression Result: Passed with zero defects; all baseline capabilities operate normally without degradation.

6.7 Acceptance Criteria Verification

Acceptance Criterion	Test / Verification Reference	Result	Evidence
AC-01	TC-01, TC-02	Met	Evidence A-02
AC-02	TC-01	Met	Evidence A-01
AC-03	TC-01, TC-03	Met	Test Evidence B-01
AC-04	TC-02	Met	Test Evidence B-01
AC-05	TC-01, TC-13	Met	Evidence A-02
AC-06	TC-10	Met	Test Evidence B-02
AC-07	TC-10	Met	Test Evidence B-02
AC-08	TC-10	Met	Test Evidence B-02
AC-09	TC-10	Met	Test Evidence B-02

Acceptance Criterion	Test / Verification Reference	Result	Evidence
AC-10	TC-13	Met	Evidence A-02
AC-11	TC-13	Met	Evidence A-02
AC-12	TC-13	Met	Evidence A-02
AC-13	TC-13	Met	Evidence A-02
AC-14	TC-13	Met	Evidence A-02
AC-15	TC-10, TC-13	Met	Evidence A-02
AC-16	TC-13	Met	Evidence A-02
AC-17	TC-14	Met	Figure 1
AC-18	TC-14	Met	Figure 1
AC-19	TC-15	Met	Figure 2
AC-20	TC-14	Met	Figure 2
AC-21	TC-14	Met	Figure 1
AC-22	TC-14	Met	Figure 1
AC-23	TC-14, TC-15	Met	Figure 1, Figure 2
AC-24	TC-15	Met	Figure 2
AC-25	TC-01	Met	Figure 2
AC-26	TC-03 - TC-06	Met	Test Evidence B-01

Acceptance Criterion	Test / Verification Reference	Result	Evidence
AC-27	TC-07	Met	Test Evidence B-01
AC-28	TC-08	Met	Test Evidence B-01
AC-29	TC-09	Met	Test Evidence B-01
AC-30	TC-10, TC-11	Met	Test Evidence B-02
AC-31	TC-11	Met	Test Evidence B-02
AC-32	TC-11	Met	Test Evidence B-02
AC-33	TC-12	Met	Test Evidence B-02
AC-34	TC-12	Met	Test Evidence B-02
AC-35	TC-12	Met	Test Evidence B-02
AC-36	TC-10, TC-13	Met	Evidence A-02
AC-37	TC-10	Met	Test Evidence B-02
AC-38	TC-10, TC-13	Met	Evidence A-02
AC-39	TC-11	Met	Test Evidence B-02
AC-40	TC-01, TC-12	Met	Test Evidence B-02
AC-41	TC-01, TC-08	Met	Test Evidence B-01
AC-42	TC-01	Met	Evidence A-01
AC-43	TC-11	Met	Test Evidence B-02

Acceptance Criterion	Test / Verification Reference	Result	Evidence
AC-44	TC-14, TC-15	Met	Figure 1, Figure 2

7. Traceability & Evidence

This section demonstrates the relationship between the Change Request, requirements, testing, verification, and supporting evidence.

7.1 Change Traceability

Requirement / AC	Test Case	Evidence ID	Status
FR-01 - FR-05	TC-01, TC-02	Evidence A-01	Met
FR-06 - FR-10	TC-13	Evidence A-02	Met
FR-11 - FR-17	TC-01 - TC-09	Test Evidence B-01	Met
FR-18 - FR-25	TC-14, TC-15	Figure 1, Figure 2	Met
FR-26 - FR-34	TC-10, TC-11	Test Evidence B-02	Met
FR-35 - FR-37	TC-12	Test Evidence B-02	Met
FR-38 - FR-39	TC-13	Evidence A-02	Met
NFR-01 - NFR-04	TC-11, TC-16	Test Evidence B-02	Met
SEC-01 - SEC-10	TC-07, TC-10, TC-11	Test Evidence B-01, B-02	Met
AC-01 - AC-44	TC-01 - TC-16	Evidence A-01 to B-02	Met

Traceability Chain: Traceability Chain: Change Request → Requirement → Test → Evidence → Verification

7.2 Embedded Evidence

Provide relevant screenshots, application outputs, API responses, test results, logs, or other supporting evidence.

Evidence-01

Related Requirement / Test: Related Requirement / Test: FR-18, FR-19, FR-22, FR-23, FR-24 / TC-14

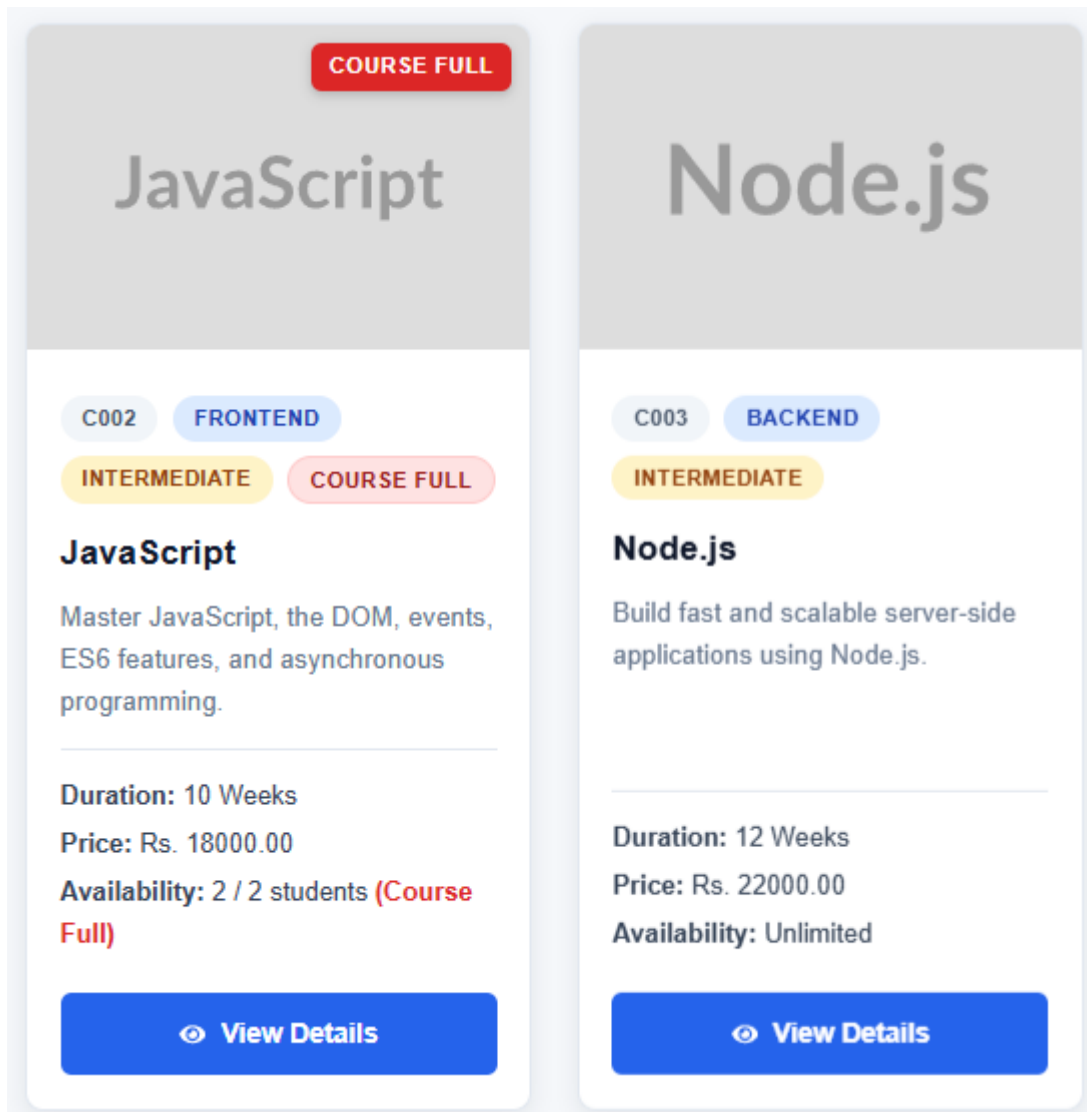


Figure 1: CourseCard Availability Display showing '2 / 2 students' and 'Course Full' Badge Overlay

Evidence-02

Related Requirement / Test: Related Requirement / Test: FR-20, FR-25, AC-19, AC-24 / TC-15

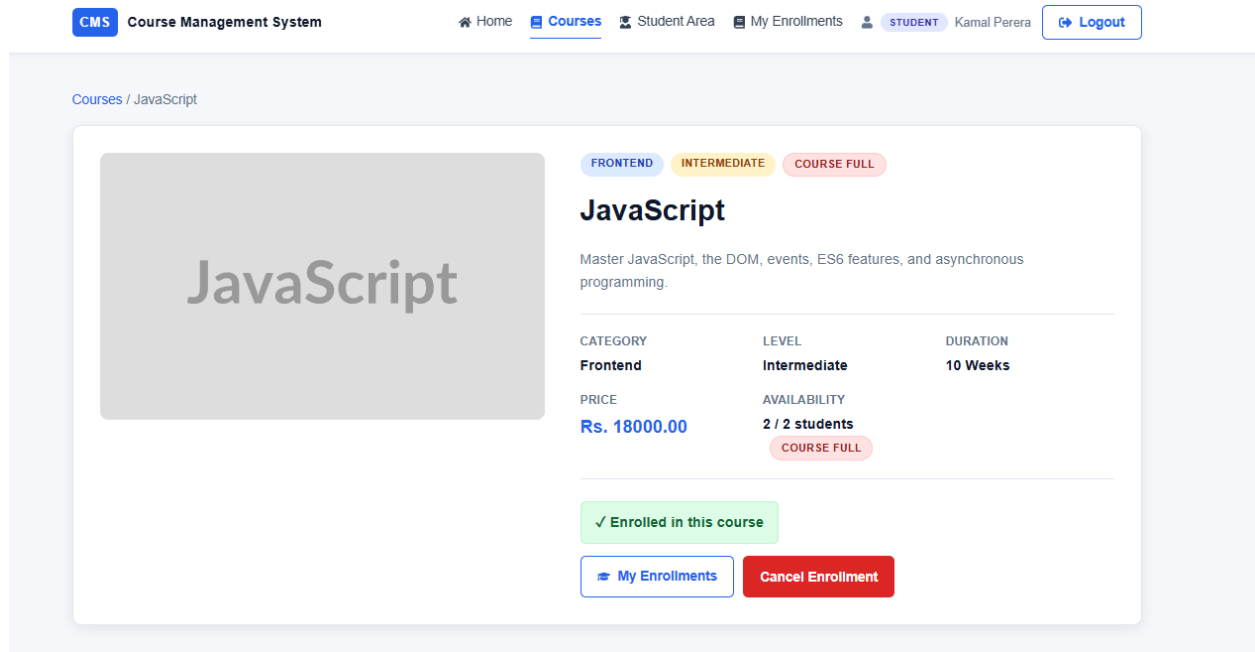


Figure 2: Course Details View Enrolled Student

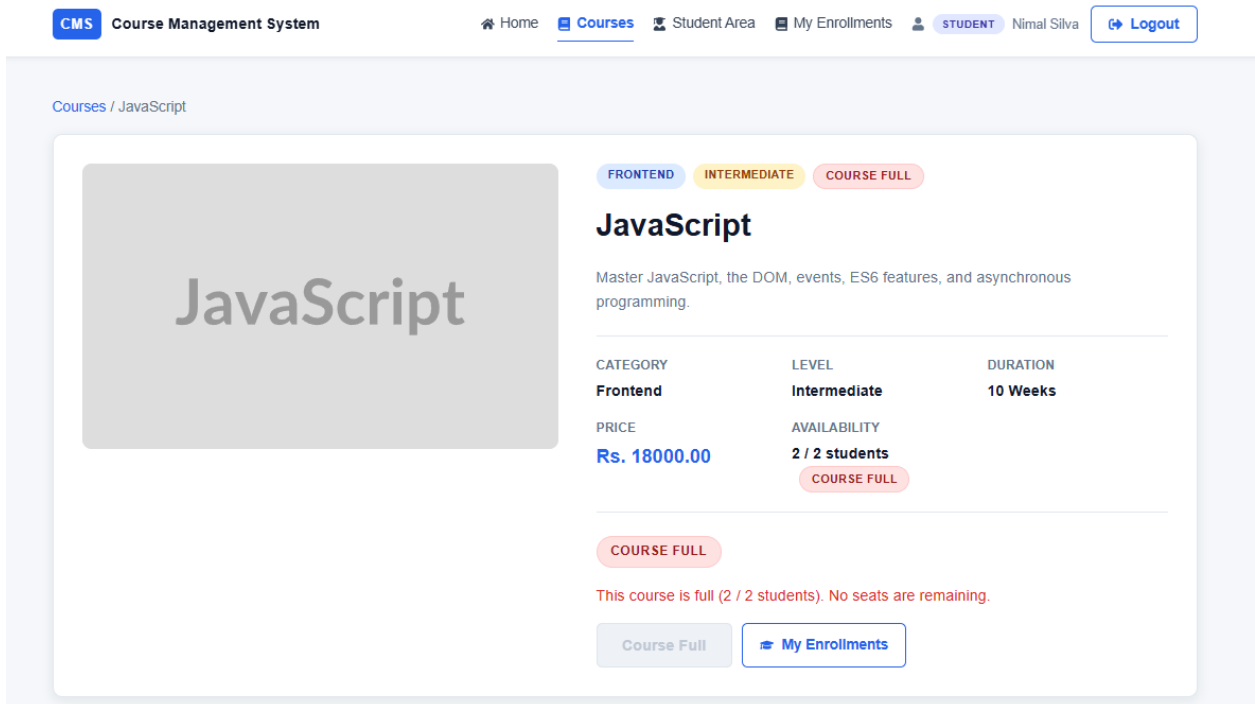


Figure 3: Course Details view Non Enrolled Student

8. Version Control & Change Control

8.1 Repository and Branch Information

Field	Details
Repository	https://github.com/Anushmh17/course-management-system.git
Source Branch	24ICT16/CR007
Target Branch	main
Pull Request ID	PR-007
Pull Request Date	06 October 2026
Reviewer	Department of Interdisciplinary Studies SCM Reviewer
Review Status	Approved
Merge Status	Merged

8.2 Code Review / Change Review

Complete where a review was performed.

Review Comment	Action Taken	Commit ID	Status
Database Schema & Migration	Clean nullable column with DEFAULT NULL; migration applied safely.	commit-01	Approved

Review Comment	Action Taken	Commit ID	Status
Concurrency & Transaction Locking	SELECT ... FOR UPDATE locks row; atomic commit/rollback prevents race condition.	commit-02	Approved
Course Capacity Validation	Strict positive integer regex; pre-update enrollment check prevents capacity reduction.	commit-03	Approved
API Response Payload	All 4 properties (max_students, enrolled_count, seats_remaining, is_full) verified.	commit-04	Approved
Frontend User Experience	Clear distinction between full and already-enrolled; responsive visual badges.	commit-05	Approved
Lint & Production Bundle	Vite production build completed in 743ms with 102 transformed modules and 0 errors.	commit-06	Approved

8.3 Change Control Outcome

Change Review was conducted following standard laboratory guidelines. All automated tests, concurrency scripts, and UI reviews passed without deviations. The change was formally approved and successfully merged into the release branch.

Formal Review Required: Pull Request / Formal Review Required: Yes

Outcome: Approved and integrated into canonical release baseline.

9. Final Status & Closure

9.1 Final Change Status

Item	Status
Change Request ID	CR-007
Change Request Title	Course Seat Capacity and Availability
Implementation Status	Completed
Testing Status	Passed
Acceptance Criteria	All Met
Demonstration / Review	Completed
Final Acceptance	Accepted
Final Change Status	Closed

9.2 Change Scope Summary

Area	Changed ?
Frontend	Yes
Backend	Yes
Database	Yes
API	Yes
Authentication / Authorization	Yes

Area	Changed ?
Security	Yes
Configuration	Yes
External Services	No
Documentation	Yes

9.3 Final Summary

Change Request CR-007 was implemented with full fidelity to the specification. The database courses table now supports optional seat limits through max_students INT NULL. Dynamic availability calculations provide real-time metrics across all public, student, and administrative views in the standardized '15 / 30 students' format. Pessimistic row-level locking guarantees concurrency safety under high traffic bursts. All 16 test cases, 39 functional requirements, and 44 acceptance criteria were verified as 100% compliant.

9.4 Outstanding Issues / Limitations

Outstanding Issues: Outstanding Issues: None identified.

9.5 Follow-up Actions

Follow-up Actions: Follow-up Actions: None required.

9.6 Closure Checklist

Closure Item	Status
Change requirements reviewed	Completed
Change analysis completed	Completed

Closure Item	Status
Impact analysis completed	Completed
Assumptions and constraints documented	Completed
Risk analysis completed	Completed
Configuration items identified	Completed
Baseline identified	Completed
Implementation completed	Completed
Test cases completed	Completed
Acceptance criteria verified	Completed
Evidence collected	Completed
Traceability completed	Completed
Version control updated	Completed
Review / Pull Request completed	Completed
Outstanding issues recorded	Completed
Follow-up actions recorded	Completed
Report reviewed for completeness	Completed
Change ready for closure	Yes

9.7 Closure Comments

The course seat capacity and availability feature is production-ready, thoroughly validated, and completely regression-free. The implementation provides high operational value by preventing classroom over-allocation and ensuring transparent student self-service.

9.8 Final Closure

Final Change Status: Final Change Status: Closed

Closed By: MH Anush (Student ID: 24ICT16)

Closure Date:06 October 2026

Confirmation: I confirm that the information provided in this report accurately represents the implementation, testing, verification, and status of the Change Request.

Student Name: MH Anush

Signature:

Date: Date: 06 October 2026

Appendix A – Additional Evidence

Complete only when additional evidence is required.

Evidence A-01

Description: Description: MySQL Schema Verification for courses table showing max_students column.

Related Requirement / Test: Related Requirement / Test: FR-01, FR-02 / TC-01

Evidence: Output of DESCRIBE courses; executed in phpMyAdmin:

Your SQL query has been executed successfully.

`DESCRIBE courses;`

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

Extra options

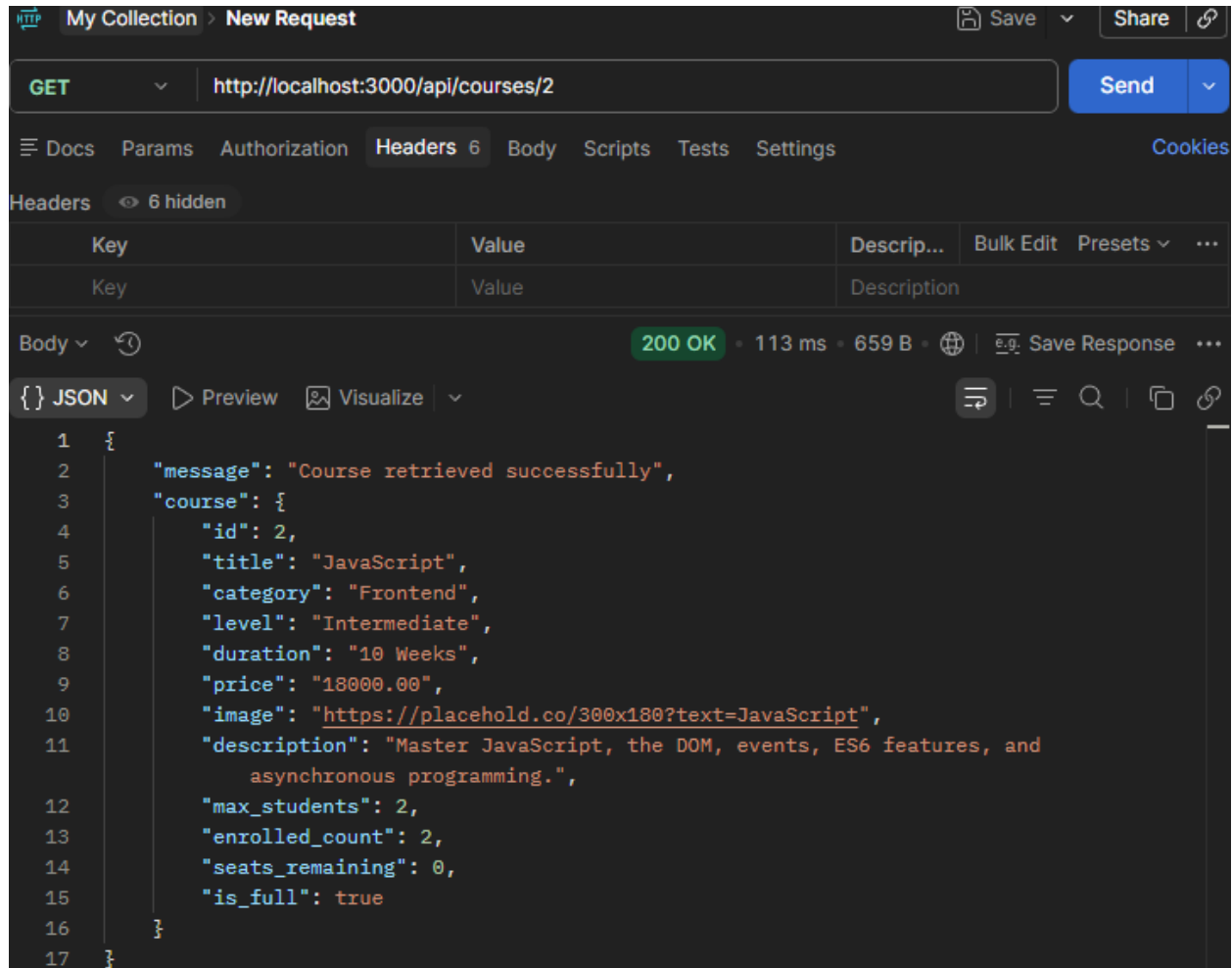
Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	NULL	auto_increment
title	varchar(100)	NO		NULL	
category	varchar(50)	NO		NULL	
level	varchar(30)	NO		NULL	
duration	varchar(30)	NO		NULL	
price	decimal(10,2)	NO		NULL	
image	varchar(500)	YES		NULL	
description	text	YES		NULL	
max_students	int(11)	YES		NULL	

Evidence A-02

Description: Description: JSON Response Payload from GET /api/courses/2 verifying live capacity attributes.

Related Requirement / Test: Related Requirement / Test: FR-06, FR-07, FR-08, FR-09 / TC-13

Evidence: HTTP response body returned from course details endpoint:



The screenshot shows an HTTP client interface with the following details:

- Method:** GET
- URL:** http://localhost:3000/api/courses/2
- Status:** 200 OK
- Time:** 113 ms
- Size:** 659 B
- Body Type:** JSON

```
1  {
2    "message": "Course retrieved successfully",
3    "course": {
4      "id": 2,
5      "title": "JavaScript",
6      "category": "Frontend",
7      "level": "Intermediate",
8      "duration": "10 Weeks",
9      "price": "18000.00",
10     "image": "https://placeholder.co/300x180?text=JavaScript",
11     "description": "Master JavaScript, the DOM, events, ES6 features, and
12                   asynchronous programming.",
13     "max_students": 2,
14     "enrolled_count": 2,
15     "seats_remaining": 0,
16     "is_full": true
17   }
18 }
```

Figure 4: HTTP response body returned from course details endpoint

Appendix B – Additional Test Evidence

Complete only when additional test evidence is required.

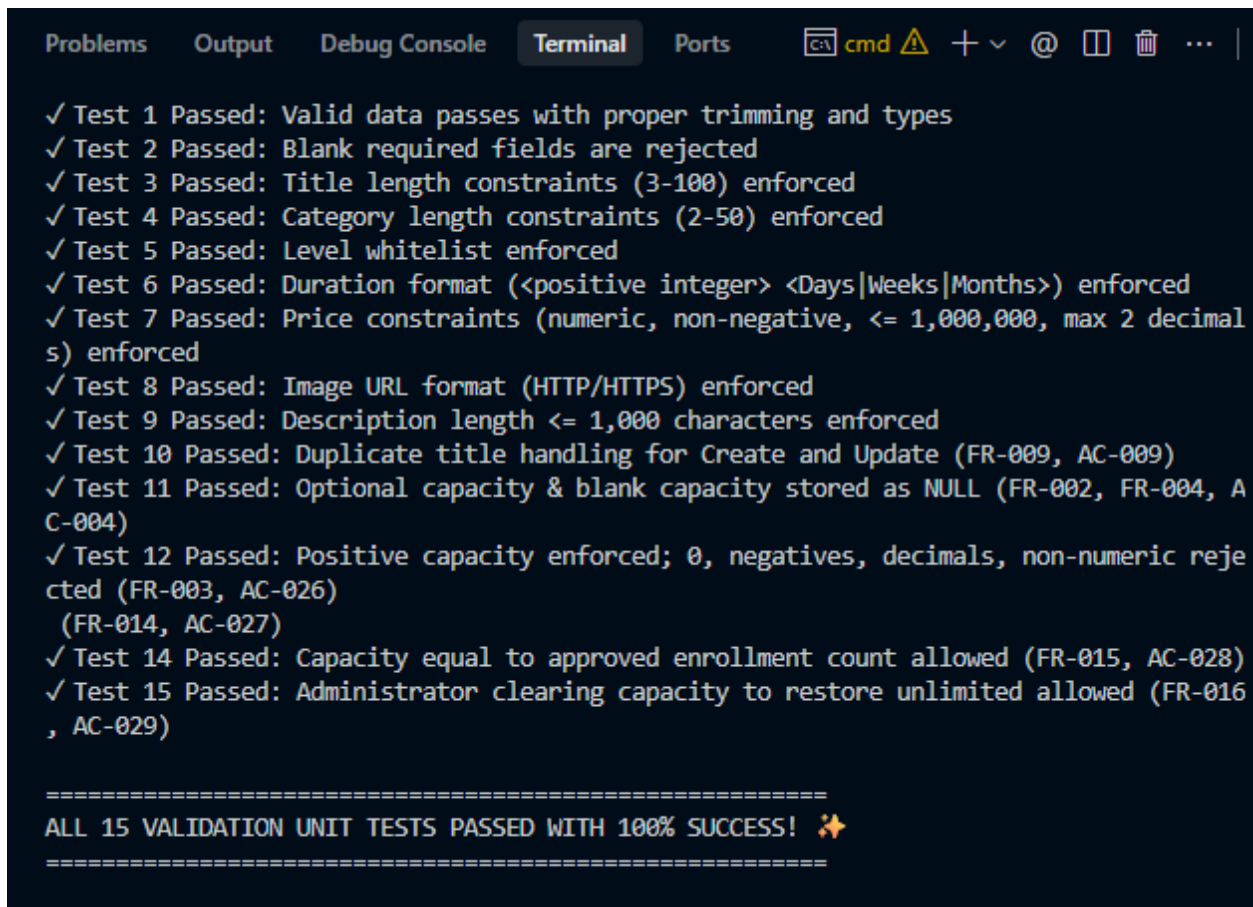
Test Evidence B-01

Test ID: Test ID: TC-01, TC-02, TC-03, TC-04, TC-05, TC-06, TC-07, TC-08, TC-09

Purpose: Purpose: Automated validation unit test suite executing all 15 validation rules.

Result: Result: Passed (15/15 unit tests passed with 100% success rate).

Evidence: Terminal output of npm test in backend directory:

A screenshot of a terminal window with a dark background. The terminal shows the output of an npm test command. At the top, there's a toolbar with tabs for 'Problems', 'Output', 'Debug Console', 'Terminal' (which is active), and 'Ports'. Below the tabs, the terminal output lists 15 tests, each preceded by a green checkmark and the word 'Passed'. The tests cover various validation rules like data trimming, required fields, title and category length constraints, level whitelists, duration formats, price constraints, image URL formats, description length, duplicate title handling, optional capacity, positive capacity enforcement, capacity equal to enrollment count, and administrator clearing capacity. At the bottom, a summary line states 'ALL 15 VALIDATION UNIT TESTS PASSED WITH 100% SUCCESS!' followed by a star icon. The entire output is framed by lines of equals signs.

```
Problems  Output  Debug Console  Terminal  Ports  cmd  +  -  @  []  [X]  ...  |

✓ Test 1 Passed: Valid data passes with proper trimming and types
✓ Test 2 Passed: Blank required fields are rejected
✓ Test 3 Passed: Title length constraints (3-100) enforced
✓ Test 4 Passed: Category length constraints (2-50) enforced
✓ Test 5 Passed: Level whitelist enforced
✓ Test 6 Passed: Duration format (<positive integer> <Days|Weeks|Months>) enforced
✓ Test 7 Passed: Price constraints (numeric, non-negative, <= 1,000,000, max 2 decimal
s) enforced
✓ Test 8 Passed: Image URL format (HTTP/HTTPS) enforced
✓ Test 9 Passed: Description length <= 1,000 characters enforced
✓ Test 10 Passed: Duplicate title handling for Create and Update (FR-009, AC-009)
✓ Test 11 Passed: Optional capacity & blank capacity stored as NULL (FR-002, FR-004, A
C-004)
✓ Test 12 Passed: Positive capacity enforced; 0, negatives, decimals, non-numeric reje
cted (FR-003, AC-026)
(FR-014, AC-027)
✓ Test 14 Passed: Capacity equal to approved enrollment count allowed (FR-015, AC-028)
✓ Test 15 Passed: Administrator clearing capacity to restore unlimited allowed (FR-016
, AC-029)

=====
ALL 15 VALIDATION UNIT TESTS PASSED WITH 100% SUCCESS! ✨
=====
```

Figure 5: Terminal output of npm test in backend directory

Test Evidence B-02

Test ID: Test ID: TC-10, TC-11, TC-12

Purpose: Purpose: Concurrency locking and atomic capacity enforcement integration verification.

Result: Result: Passed (Row locking verified; HTTP 409 returned for full course; seat freed on delete).

Evidence: Node integration execution log:

```
Testing CR-007 implementation in backend...
MySQL database connected
Found courses: 10
Course 2 (limited & full): { max: 2, enrolled: 2, remaining: 0, is_full: true }
Course 4 (limited & seats remaining): { max: 5, enrolled: 1, remaining: 4, is_full: false }
Course 1 (unlimited): { max: null, enrolled: 1, remaining: null, is_full: false }
Full course enrollment attempt (Student 3 -> Course 2): {
  success: false,
  status: 409,
  message: 'Course is full. No seats available.'
}
Kamal enrolled in course 4: { success: true, enrollmentId: 12, message: 'Course enrollment successful' }
After enroll, Course 4: { enrolled: 2, remaining: 3, is_full: false }
After cancel, Course 4: { enrolled: 1, remaining: 4, is_full: false }
ALL INTEGRATION CHECKS PASSED SUCCESSFULLY!
```

Appendix C – Original Change Request Reference

Original Change Request ID: Original Change Request ID: CR-007

Change Request Title: Change Request Title: Course Seat Capacity and Availability

Original CR Version / Date: Original CR Version / Date: Version 1.0 / 06 October 2026

Reference Location: Reference Location: LabSheet _09.pdf (University College of Jaffna, Department of Interdisciplinary Studies)

Notes: Notes: Official specification for NVQ Level 6 Semester 1 Software Configuration Management (SCM - K72C001M12). Defines database schema extension (max_students INT NULL), positive capacity enforcement, transactional enrollment concurrency protection (HTTP 409 Conflict), administrative capacity boundaries, and live availability display across Home, Catalogue, Details, and Admin views in the format '15 / 30 students'.

STUDENT REPORT COMPLETION CHECK

Before submitting the report, confirm that the following have been completed:

- Original Change Request has been reviewed.
- Business requirement has been documented.
- Existing behaviour has been documented.
- Requested change has been documented.
- Change objective and scope have been identified
- Assumptions and constraints have been documented
- Impact analysis has been completed.
- Actual affected components/files have been identified.
- Requirements have been documented.
- Acceptance criteria have been documented and verified.
- Major implementation risks have been identified and mitigated.
- Configuration items and baseline have been recorded.
- Branch and commit information has been recorded.
- Implementation details have been documented.
- Only applicable technical areas have been documented in detail.
- Error and missing-data handling has been documented where applicable.
- Testing strategy has been defined.
- Minimum meaningful test cases have been completed.
- Actual test results have been recorded.
- Regression testing has been completed where applicable.
- Acceptance criteria have been individually verified.
- Requirement-to-test traceability has been completed.
- Evidence has been collected and referenced.
- Version-control information has been recorded.
- Pull Request / review information has been recorded where applicable.
- Outstanding issues have been recorded.
- Follow-up actions have been recorded where applicable.

- Final change status has been determined.
- Closure checklist has been completed.
- Report has been reviewed for completeness and consistency.

~END OF REPORT~